

Guia: Nginx em container com SSL wildcard, múltiplos hosts e reaproveitamento do certificado

Resumo

Este tutorial cria um Nginx em container usando `docker compose`, com estrutura organizada em `/opt/nginx`.

O ambiente terá diretórios separados para:

```
/opt/nginx/conf.d
```

```
/opt/nginx/ssl
```

```
/opt/nginx/ca
```

```
/opt/nginx/html
```

```
/opt/nginx/logs
```

O Nginx responderá por dois hosts:

```
docker-hm.jmsalles.homelab.br
```

```
pdf.jmsalles.homelab.br
```

O host `docker-hm.jmsalles.homelab.br` exibirá uma página HTML local do próprio Nginx.

O host `pdf.jmsalles.homelab.br` funcionará como proxy reverso para uma aplicação rodando na porta `8080` da máquina Docker.

Também será criado um certificado wildcard para o domínio:

```
*.jmsalles.homelab.br
```

Esse certificado poderá ser reaproveitado em outras máquinas do homelab com Nginx em container.

Sumário

- Visão geral do cenário
- Criar estrutura de diretórios
- Criar autoridade certificadora local
- Criar certificado wildcard
- Criar página HTML de teste
- Criar configuração do host docker-hm
- Criar configuração do host pdf
- Criar docker-compose.yml
- Ajustar SELinux
- Liberar firewall
- Subir o container
- Validar Nginx e SSL
- Ajustar DNS ou hosts
- Testar backend da aplicação PDF
- Reaproveitar certificados em outra máquina
- Comandos de operação
- Checklist final
- Estrutura final esperada

Visão geral do cenário

Estrutura principal:

```
/opt/nginx/  
├── docker-compose.yml  
├── ca/  
├── conf.d/  
├── html/  
├── logs/  
└── ssl/
```

Hosts atendidos pelo Nginx:

```
docker-hm.jmsalles.homelab.br
```

```
pdf.jmsalles.homelab.br
```

Certificado SSL utilizado:

```
*.jmsalles.homelab.br
```

Também será incluído o domínio raiz:

```
jmsalles.homelab.br
```

Fluxo do host **docker-hm**:

```
Cliente -> https://docker-hm.jmsalles.homelab.br -> Nginx container -> página  
HTML local
```

Fluxo do host **pdf**:

```
Cliente -> https://pdf.jmsalles.homelab.br -> Nginx container ->  
http://host.docker.internal:8080
```

Observação importante: dentro do container Nginx, **127.0.0.1** aponta para o próprio container, não para a máquina Docker. Por isso, para acessar uma aplicação rodando no host Docker, será usado:

```
host.docker.internal
```

Criar estrutura de diretórios

Crie os diretórios principais:

```
sudo mkdir -p /opt/nginx/{conf.d,ssl,html,logs,ca}
```

Ajuste o dono do diretório:

```
sudo chown -R $USER:$USER /opt/nginx
```

Acesse o diretório:

```
cd /opt/nginx
```

Valide:

```
ls -l /opt/nginx
```

Criar autoridade certificadora local

Será criada uma CA local para assinar o certificado wildcard do homelab.

Crie a chave privada da CA:

```
openssl genrsa -out /opt/nginx/ca/jmsalles-homelab-rootCA.key 4096
```

Crie o certificado da CA local:

```
openssl req -x509 -new -nodes \  
-key /opt/nginx/ca/jmsalles-homelab-rootCA.key \  
-sha256 \  
-days 3650 \  
-out /opt/nginx/ca/jmsalles-homelab-rootCA.crt \  
-subj "/C=BR/ST=RJ/L=Mesquita/O=JMSalles Homelab/OU=Homelab/CN=JMSalles  
Homelab Root CA"
```

Valide os arquivos criados:

```
ls -l /opt/nginx/ca
```

Criar certificado wildcard

Crie a chave privada do certificado wildcard:

```
openssl genrsa -out /opt/nginx/ssl/jmsalles.homelab.br.key 2048
```

Crie o arquivo de configuração do certificado:

```
vim /opt/nginx/ssl/jmsalles.homelab.br.cnf
```

Conteúdo:

```
[req]
default_bits      = 2048
prompt            = no
default_md         = sha256
distinguished_name = dn
req_extensions     = v3_req
```

```
[dn]
C  = BR
ST = RJ
L  = Mesquita
O  = JMSalles Homelab
OU = Homelab
CN = *.jmsalles.homelab.br
```

```
[v3_req]
subjectAltName = @alt_names
```

```
[alt_names]
DNS.1 = *.jmsalles.homelab.br
DNS.2 = jmsalles.homelab.br
DNS.3 = docker-hm.jmsalles.homelab.br
DNS.4 = pdf.jmsalles.homelab.br
```

Gere o CSR:

```
openssl req -new \
  -key /opt/nginx/ssl/jmsalles.homelab.br.key \
  -out /opt/nginx/ssl/jmsalles.homelab.br.csr \
  -config /opt/nginx/ssl/jmsalles.homelab.br.cnf
```

Assine o certificado usando a CA local:

```
openssl x509 -req \
  -in /opt/nginx/ssl/jmsalles.homelab.br.csr \
  -CA /opt/nginx/ca/jmsalles-homelab-rootCA.crt \
  -CAkey /opt/nginx/ca/jmsalles-homelab-rootCA.key \
  -CAcreateserial \
  -out /opt/nginx/ssl/jmsalles.homelab.br.crt \
  -days 825 \
  -sha256 \
  -extensions v3_req \
  -extfile /opt/nginx/ssl/jmsalles.homelab.br.cnf
```

Ajuste as permissões:

```
chmod 600 /opt/nginx/ssl/jmsalles.homelab.br.key
```

```
chmod 644 /opt/nginx/ssl/jmsalles.homelab.br.crt
```

```
chmod 600 /opt/nginx/ca/jmsalles-homelab-rootCA.key
```

```
chmod 644 /opt/nginx/ca/jmsalles-homelab-rootCA.crt
```

Valide o certificado:

```
openssl x509 -in /opt/nginx/ssl/jmsalles.homelab.br.crt -noout -subject -issuer  
-dates
```

Valide os nomes DNS do certificado:

```
openssl x509 -in /opt/nginx/ssl/jmsalles.homelab.br.crt -noout -text | grep -A1  
"Subject Alternative Name"
```

O retorno esperado deve conter:

```
DNS:*.jmsalles.homelab.br
```

```
DNS:jmsalles.homelab.br
```

```
DNS:docker-hm.jmsalles.homelab.br
```

```
DNS:pdf.jmsalles.homelab.br
```

Criar página HTML de teste

Essa página será usada pelo host `docker-hm.jmsalles.homelab.br`.

Crie o arquivo:

```
vim /opt/nginx/html/index.html
```

Conteúdo:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Docker HM - Nginx</title>
</head>
<body>
  <h1>Nginx em container funcionando</h1>
  <p>Host: docker-hm.jmsalles.homelab.br</p>
  <p>Ambiente: Homelab JMSalles</p>
</body>
</html>
```

Criar configuração do host docker-hm

Crie o arquivo:

```
vim /opt/nginx/conf.d/docker-hm.jmsalles.homelab.br.conf
```

Conteúdo:

```
server {
    listen 80;
    server_name docker-hm.jmsalles.homelab.br;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name docker-hm.jmsalles.homelab.br;

    ssl_certificate      /etc/nginx/ssl/jmsalles.homelab.br.crt;
    ssl_certificate_key  /etc/nginx/ssl/jmsalles.homelab.br.key;

    access_log /var/log/nginx/docker-hm.jmsalles.homelab.br.access.log;
    error_log  /var/log/nginx/docker-hm.jmsalles.homelab.br.error.log;

    root /usr/share/nginx/html;
    index index.html;
```

```
location / {  
    try_files $uri $uri/ =404;  
}  
}
```

Esse arquivo faz o host abaixo responder com uma página HTML local:

```
https://docker-hm.jmsalles.homelab.br
```

Criar configuração do host pdf

Crie o arquivo:

```
vim /opt/nginx/conf.d/pdf.jmsalles.homelab.br.conf
```

Conteúdo:

```
server {  
    listen 80;  
    server_name pdf.jmsalles.homelab.br;  
  
    return 301 https://$host$request_uri;  
}  
  
server {  
    listen 443 ssl;  
    server_name pdf.jmsalles.homelab.br;  
  
    ssl_certificate      /etc/nginx/ssl/jmsalles.homelab.br.crt;  
    ssl_certificate_key  /etc/nginx/ssl/jmsalles.homelab.br.key;  
  
    access_log /var/log/nginx/pdf.jmsalles.homelab.br.access.log;  
    error_log  /var/log/nginx/pdf.jmsalles.homelab.br.error.log;  
  
    location / {  
        proxy_pass http://host.docker.internal:8080;  
  
        proxy_http_version 1.1;  
  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto https;  
        proxy_set_header X-Forwarded-Host $host;  
        proxy_set_header X-Forwarded-Port 443;  
    }  
}
```



```
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }
}
```

Esse arquivo faz o host abaixo funcionar como proxy reverso:

```
https://pdf.jmsalles.homelab.br
```

O destino do proxy será:

```
http://host.docker.internal:8080
```

Criar docker-compose.yml

Crie o arquivo:

```
vim /opt/nginx/docker-compose.yml
```

Conteúdo:

```
services:
  nginx:
    image: nginx:stable-alpine
    container_name: nginx-web
    restart: unless-stopped
    extra_hosts:
      - "host.docker.internal:host-gateway"
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /opt/nginx/conf.d:/etc/nginx/conf.d:ro
      - /opt/nginx/ssl:/etc/nginx/ssl:ro
      - /opt/nginx/html:/usr/share/nginx/html:ro
      - /opt/nginx/logs:/var/log/nginx
    networks:
      - nginx-net

networks:
  nginx-net:
    driver: bridge
```

O item abaixo é importante para o container conseguir acessar serviços que estão rodando diretamente no host Docker:

```
extra_hosts:  
  - "host.docker.internal:host-gateway"
```

Ajustar SELinux

Se a máquina for Rocky Linux, RHEL, AlmaLinux ou Oracle Linux com SELinux em modo enforcing, aplique o contexto correto nos diretórios montados:

```
sudo chcon -Rt virt_sandbox_file_t /opt/nginx
```

Valide o status do SELinux:

```
getenforce
```

Liberar firewall

Se estiver usando `firewalld`, libere HTTP e HTTPS:

```
sudo firewall-cmd --permanent --add-service=http
```

```
sudo firewall-cmd --permanent --add-service=https
```

Recarregue o firewall:

```
sudo firewall-cmd --reload
```

Valide:

```
sudo firewall-cmd --list-services
```

Subir o container

Acesse o diretório:

```
cd /opt/nginx
```

Suba o container:

```
docker compose -f docker-compose.yml up -d
```

Valide se o container está rodando:

```
docker ps
```

Valide os logs:

```
docker compose -f /opt/nginx/docker-compose.yml logs -f nginx
```

Validar configuração do Nginx

Teste a configuração dentro do container:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx nginx -t
```

Resultado esperado:

```
syntax is ok  
test is successful
```

Recarregue o Nginx:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx nginx -s reload
```

Ajustar DNS ou hosts

Os dois hosts precisam apontar para o IP da máquina onde está rodando o container Nginx.

No DNS interno, crie registros tipo A:

```
docker-hm.jmsalles.homelab.br -> IP_DA_MAQUINA_DOCKER_HM
```

```
pdf.jmsalles.homelab.br -> IP_DA_MAQUINA_DOCKER_HM
```

Para teste local temporário em um cliente Linux, edite o `/etc/hosts`:

```
vim /etc/hosts
```

Exemplo:

```
192.168.31.X docker-hm.jmsalles.homelab.br pdf.jmsalles.homelab.br
```

No Windows, o arquivo equivalente fica em:

```
C:\Windows\System32\drivers\etc\hosts
```

Valide a resolução DNS:

```
getent hosts docker-hm.jmsalles.homelab.br
```

```
getent hosts pdf.jmsalles.homelab.br
```

Testar redirecionamento HTTP para HTTPS

Teste o host `docker-hm`:

```
curl -I http://docker-hm.jmsalles.homelab.br
```

Teste o host `pdf`:

```
curl -I http://pdf.jmsalles.homelab.br
```

O retorno esperado deve ser redirecionamento para HTTPS.

Testar acesso HTTPS

Teste o host `docker-hm`:

```
curl -k -I https://docker-hm.jmsalles.homelab.br
```

Teste o host **pdf**:

```
curl -k -I https://pdf.jmsalles.homelab.br
```

O parâmetro **-k** é usado porque o certificado foi assinado por uma CA local. Depois de importar a CA nos clientes, o navegador passará a confiar no certificado.

Testar backend da aplicação PDF

Antes de validar pelo Nginx, confirme se a aplicação PDF está rodando na porta **8080** da própria máquina Docker:

```
curl -I http://127.0.0.1:8080
```

Também valide se a porta está escutando:

```
ss -tulnp | grep 8080
```

Agora teste se o container Nginx consegue acessar o backend:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx wget -S -O-  
http://host.docker.internal:8080
```

Se esse teste falhar, o problema não está no certificado nem no **server_name**. O problema estará na comunicação entre o container Nginx e a aplicação da porta **8080**.

Validar certificado apresentado pelo Nginx

Valide o certificado do host **docker-hm**:

```
openssl s_client -connect docker-hm.jmsalles.homelab.br:443 -servername docker-  
hm.jmsalles.homelab.br </dev/null 2>/dev/null | openssl x509 -noout -subject -  
issuer -dates
```

Valide o certificado do host **pdf**:

```
openssl s_client -connect pdf.jmsalles.homelab.br:443 -servername  
pdf.jmsalles.homelab.br </dev/null 2>/dev/null | openssl x509 -noout -subject -  
issuer -dates
```

Importar a CA local nos clientes

Para o navegador confiar no certificado, importe o arquivo da CA local:

```
/opt/nginx/ca/jmsalles-homelab-rootCA.crt
```

No Windows, importe em:

Autoridades de Certificação Raiz Confiáveis

No Linux, em distribuições baseadas em RHEL, copie a CA para o diretório de certificados confiáveis:

```
sudo cp /opt/nginx/ca/jmsalles-homelab-rootCA.crt /etc/pki/ca-  
trust/source/anchors/
```

Atualize a trust store:

```
sudo update-ca-trust
```

Em distribuições baseadas em Debian/Ubuntu:

```
sudo cp /opt/nginx/ca/jmsalles-homelab-rootCA.crt /usr/local/share/ca-  
certificates/jmsalles-homelab-rootCA.crt
```

```
sudo update-ca-certificates
```

Reaproveitar certificados em outra máquina com Nginx em container

Como o certificado criado é wildcard para:

```
*.jmsalles.homelab.br
```

ele pode ser usado em outras máquinas do seu homelab para responder por novos hosts, por exemplo:

```
zabbix.jmsalles.homelab.br
```

```
teampass.jmsalles.homelab.br
```

```
portainer.jmsalles.homelab.br
```

```
grafana.jmsalles.homelab.br
```

Todos esses hosts estão dentro do domínio:

```
jmsalles.homelab.br
```

Por isso podem usar o mesmo certificado wildcard.

Arquivos necessários para reaproveitamento

Na máquina original, os arquivos principais são:

```
/opt/nginx/ssl/jmsalles.homelab.br.crt
```

```
/opt/nginx/ssl/jmsalles.homelab.br.key
```

```
/opt/nginx/ca/jmsalles-homelab-rootCA.crt
```

O arquivo `.crt` é o certificado wildcard.

O arquivo `.key` é a chave privada do certificado wildcard.

O arquivo da CA local é usado para importar nos clientes e fazer os navegadores confiarem no certificado.

Não é recomendado copiar a chave privada da CA para outras máquinas:

```
/opt/nginx/ca/jmsalles-homelab-rootCA.key
```

Essa chave só deve ficar na máquina onde você pretende emitir novos certificados.

Criar estrutura na nova máquina

Na nova máquina que também terá Nginx em container, crie a estrutura:

```
sudo mkdir -p /opt/nginx/{conf.d,ssl,html,logs,ca}
```

```
sudo chown -R $USER:$USER /opt/nginx
```

```
cd /opt/nginx
```

Copiar os certificados para a nova máquina

Na nova máquina, copie os arquivos a partir da máquina original.

Exemplo usando `scp` com IP da máquina original:

```
scp usuario@IP_DA_MAUQUINA_ORIGINAL:/opt/nginx/ssl/jmsalles.homelab.br.crt  
/opt/nginx/ssl/
```

```
scp usuario@IP_DA_MAUQUINA_ORIGINAL:/opt/nginx/ssl/jmsalles.homelab.br.key  
/opt/nginx/ssl/
```

```
scp usuario@IP_DA_MAUQUINA_ORIGINAL:/opt/nginx/ca/jmsalles-homelab-rootCA.crt  
/opt/nginx/ca/
```

Exemplo usando o host original:

```
scp usuario@docker-hm.jmsalles.homelab.br:/opt/nginx/ssl/jmsalles.homelab.br.crt  
/opt/nginx/ssl/
```



```
scp usuario@docker-hm.jmsalles.homelab.br:/opt/nginx/ssl/jmsalles.homelab.br.key  
/opt/nginx/ssl/
```

```
scp usuario@docker-hm.jmsalles.homelab.br:/opt/nginx/ca/jmsalles-homelab-  
rootCA.crt /opt/nginx/ca/
```

Ajuste as permissões na nova máquina:

```
chmod 600 /opt/nginx/ssl/jmsalles.homelab.br.key
```

```
chmod 644 /opt/nginx/ssl/jmsalles.homelab.br.crt
```

```
chmod 644 /opt/nginx/ca/jmsalles-homelab-rootCA.crt
```

Valide o certificado copiado:

```
openssl x509 -in /opt/nginx/ssl/jmsalles.homelab.br.crt -noout -subject -issuer  
-dates
```

Valide o wildcard:

```
openssl x509 -in /opt/nginx/ssl/jmsalles.homelab.br.crt -noout -text | grep -A1  
"Subject Alternative Name"
```

Criar docker-compose.yml na nova máquina

Crie o arquivo:

```
vim /opt/nginx/docker-compose.yml
```

Conteúdo:

```
services:  
  nginx:  
    image: nginx:stable-alpine
```

```
container_name: nginx-web
restart: unless-stopped
extra_hosts:
  - "host.docker.internal:host-gateway"
ports:
  - "80:80"
  - "443:443"
volumes:
  - /opt/nginx/conf.d:/etc/nginx/conf.d:ro
  - /opt/nginx/ssl:/etc/nginx/ssl:ro
  - /opt/nginx/html:/usr/share/nginx/html:ro
  - /opt/nginx/logs:/var/log/nginx
networks:
  - nginx-net

networks:
  nginx-net:
    driver: bridge
```

Exemplo de novo host na outra máquina

Exemplo: a nova máquina irá responder por:

```
zabbix.jmsalles.homelab.br
```

Crie uma página de teste:

```
vim /opt/nginx/html/index.html
```

Conteúdo:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Zabbix Homelab</title>
</head>
<body>
  <h1>Nginx em container funcionando</h1>
  <p>Host: zabbix.jmsalles.homelab.br</p>
  <p>Certificado wildcard reaproveitado com sucesso.</p>
</body>
</html>
```

Crie a configuração do host:

```
vim /opt/nginx/conf.d/zabbix.jmsalles.homelab.br.conf
```

Conteúdo para página local:

```
server {
    listen 80;
    server_name zabbix.jmsalles.homelab.br;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name zabbix.jmsalles.homelab.br;

    ssl_certificate      /etc/nginx/ssl/jmsalles.homelab.br.crt;
    ssl_certificate_key  /etc/nginx/ssl/jmsalles.homelab.br.key;

    access_log /var/log/nginx/zabbix.jmsalles.homelab.br.access.log;
    error_log  /var/log/nginx/zabbix.jmsalles.homelab.br.error.log;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Suba o Nginx na nova máquina:

```
cd /opt/nginx
```

```
docker compose -f docker-compose.yml up -d
```

Teste a configuração:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx nginx -t
```

Recarregue o Nginx:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx nginx -s reload
```

No DNS interno, aponte o host para o IP da nova máquina:

```
zabbix.jmsalles.homelab.br -> IP_DA_NOVA_MAQUINA
```

Teste:

```
curl -k -I https://zabbix.jmsalles.homelab.br
```

Exemplo de proxy reverso na outra máquina

Se a nova máquina tiver uma aplicação local na porta **8080**, crie um novo arquivo:

```
vim /opt/nginx/conf.d/app.jmsalles.homelab.br.conf
```

Conteúdo:

```
server {
    listen 80;
    server_name app.jmsalles.homelab.br;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name app.jmsalles.homelab.br;

    ssl_certificate      /etc/nginx/ssl/jmsalles.homelab.br.crt;
    ssl_certificate_key  /etc/nginx/ssl/jmsalles.homelab.br.key;

    access_log /var/log/nginx/app.jmsalles.homelab.br.access.log;
    error_log  /var/log/nginx/app.jmsalles.homelab.br.error.log;

    location / {
        proxy_pass http://host.docker.internal:8080;

        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

```
    proxy_set_header X-Forwarded-Host $host;
    proxy_set_header X-Forwarded-Port 443;

    proxy_connect_timeout 60s;
    proxy_send_timeout 60s;
    proxy_read_timeout 60s;
}
}
```

No DNS interno:

```
app.jmsalles.homelab.br -> IP_DA_NOVA_MAQUINA
```

Teste:

```
curl -k -I https://app.jmsalles.homelab.br
```

Comandos de operação

Parar o ambiente:

```
cd /opt/nginx
```

```
docker compose -f docker-compose.yml down
```

Subir o ambiente:

```
cd /opt/nginx
```

```
docker compose -f docker-compose.yml up -d
```

Reiniciar o container:

```
docker compose -f /opt/nginx/docker-compose.yml restart nginx
```

Ver logs:

```
docker compose -f /opt/nginx/docker-compose.yml logs -f nginx
```

Testar configuração:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx nginx -t
```

Recarregar configuração:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx nginx -s reload
```

Acessar o shell do container:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx sh
```

Listar arquivos de configuração carregados:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx ls -l  
/etc/nginx/conf.d
```

Listar certificados dentro do container:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx ls -l /etc/nginx/ssl
```

Checklist final

Valide diretórios:

```
ls -l /opt/nginx
```

Valide configurações:

```
ls -l /opt/nginx/conf.d
```

Valide certificados:

```
ls -l /opt/nginx/ssl
```

Valide CA:

```
ls -l /opt/nginx/ca
```

Valide container:

```
docker ps
```

Valide portas:

```
ss -tulnp | egrep ' :80| :443 '
```

Valide Nginx:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx nginx -t
```

Valide host `docker-hm`:

```
curl -k -I https://docker-hm.jmsalles.homelab.br
```

Valide host `pdf`:

```
curl -k -I https://pdf.jmsalles.homelab.br
```

Valide aplicação PDF diretamente:

```
curl -I http://127.0.0.1:8080
```

Valide acesso do container até a aplicação PDF:

```
docker compose -f /opt/nginx/docker-compose.yml exec nginx wget -S -O-  
http://host.docker.internal:8080
```

Valide certificado apresentado pelo host **docker-hm**:

```
openssl s_client -connect docker-hm.jmsalles.homelab.br:443 -servername docker-hm.jmsalles.homelab.br </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -dates
```

Valide certificado apresentado pelo host **pdf**:

```
openssl s_client -connect pdf.jmsalles.homelab.br:443 -servername pdf.jmsalles.homelab.br </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -dates
```

Estrutura final esperada

```
/opt/nginx/
├── docker-compose.yml
├── ca/
│   ├── jmsalles-homelab-rootCA.crt
│   ├── jmsalles-homelab-rootCA.key
│   └── jmsalles-homelab-rootCA.srl
├── conf.d/
│   ├── docker-hm.jmsalles.homelab.br.conf
│   └── pdf.jmsalles.homelab.br.conf
├── html/
│   └── index.html
├── logs/
└── ssl/
    ├── jmsalles.homelab.br.cnf
    ├── jmsalles.homelab.br.crt
    ├── jmsalles.homelab.br.csr
    └── jmsalles.homelab.br.key
```

Resultado esperado

Ao final, o Nginx em container responderá em HTTPS pelos dois hosts:

```
https://docker-hm.jmsalles.homelab.br
```

```
https://pdf.jmsalles.homelab.br
```

O host **docker-hm.jmsalles.homelab.br** exibirá uma página HTML local.

O host `pdf.jmsalles.homelab.br` encaminhará as requisições para a aplicação PDF rodando na porta `8080` da máquina Docker.

O certificado wildcard poderá ser reaproveitado em outras máquinas do homelab para novos hosts dentro do domínio:

```
jmsalles.homelab.br
```

Criado por Jeferson Salles LinkedIn: <https://www.linkedin.com/in/jmsalles/> E-mail: jefersonmattossalles@gmail.com GitHub: <https://github.com/jmsalles/>